

## 1 Общие сведения

Размеры и выравнивания базовых типов данных C++:

| Имя типа    | Размер в словах процессора | Количество значимых бит | Выравнивание типа данных в памяти |
|-------------|----------------------------|-------------------------|-----------------------------------|
| char        | 1                          | 8                       | 1                                 |
| short       | 1                          | 16                      | 1                                 |
| int         | 1                          | 32                      | 1                                 |
| long        | 2                          | 64                      | 2                                 |
| void *      | 1                          | 32                      | 1                                 |
| float       | 1                          | 32                      | 1                                 |
| double      | 2                          | 64                      | 2                                 |
| long double | 2                          | 64                      | 2                                 |

Примечание: размер типа данных совпадает с выравниванием данного типа в памяти.

Архитектурные особенности:

Обращение в память для чтения 64-х битного длинного слова возможно только по четному адресу. Вызов функции возможен, только когда регистр указатель стека имеет четное значение.

## 2 Передаваемые параметры функций

Все параметры функции передаются через стек. Для базовых типов данных в стеке передается сам параметр. Для структурных типов данных (структуры, классы, объединения) в стеке передается указатель на структуру.

Компилятор может порождать дополнительные, неявные параметры для функций. Для функций, возвращающих структурный тип данных, добавляется параметр - указатель на область стека в которой располагается структура. Если у функции имеется такой параметр, то он предшествует всем остальным явным и неявным параметрам. Для функций членов класса компилятор порождает неявный параметр - указатель на объект класса (this). Такой параметр предшествует всем явным параметрам. Если у функции имеется два неявных параметра - указатель на возвращаемую структуру и указатель this, то первым параметром является указатель на возвращаемую структуру, вторым указатель this, затем идут явные параметры.

Параметры размещаются в стеке начиная с последнего, т.е. первый параметр имеет наибольший адрес, последний параметр имеет наименьший адрес.

Выравнивание параметров в памяти соответствует обычному выравниванию переменных такого типа. При этом в стеке могут образовываться неиспользуемые места. Такие неиспользуемые места имеют размер 1 слово и располагаются по четным адресам. Поскольку, при выполнении инструкции перехода к подпрограмме, указатель стека должен иметь четное значение,

выравнивание параметров производится относительно места в стеке, где располагаются PC и PSW при переходе к подпрограмме.

Примеры:

1. Расположение в стеке параметров функции  $f(\text{int } a, \text{long } b, \text{int } c)$  на момент входа в функцию (адрес растет вверх):

|                         |
|-------------------------|
|                         |
| Текущий указатель стека |
| PSW                     |
| PC                      |
| a                       |
| Неиспользуемое место    |
| Старшая часть b         |
| Младшая часть b         |
| c                       |

2. Расположение в стеке параметров функции  $g(\text{int } a, \text{int } b, \text{long } c, \text{int } d)$  на момент входа в функцию (адрес растет вверх):

|                         |
|-------------------------|
|                         |
| Текущий указатель стека |
| PSW                     |
| PC                      |
| a                       |
| b                       |
| Старшая часть c         |
| Младшая часть c         |
| d                       |

Замечание: Следует обратить внимание, что параметры размером в 1 слово всегда выравниваются на больший адрес и соответственно пустые места всегда занимают часть длинного слова с меньшим адресом. Такое выравнивание гарантирует, что расположение параметра в стеке не зависит от последующих параметров. Это особенно важно для функций с переменным числом параметров, когда функция обращаясь к своему параметру не знает о типе и вообще о существовании следующего параметра.

Алгоритм 1. Вычисление расположения параметров функции в стеке и занимаемого ими размера:

1. Установить переменную РАЗМЕР в 0. Эта переменная содержит размер, занимаемый параметрами в стеке.
2. Взять очередной параметр из списка параметров в направлении от первого параметра к последнему. Если список параметров полностью просмотрен, то закончить работу.
3. Если параметр имеет размер 2 слова (long), то принять в качестве смещения параметра  $\text{РАЗМЕР} + 2$ ;  $\text{РАЗМЕР} += 2$ . Перейти к шагу 2.

4. В данном случае параметр имеет размер 1 слово ( int ). Принять в качестве смещения параметра РАЗМЕР + 1; РАЗМЕР += 1. После этого взять следующий параметр из списка. Если список полностью просмотрен, то закончить работу.
5. Если следующий параметр имеет размер 1 ( ситуация int,int ), то в качестве смещения второго параметра взять РАЗМЕР + 1; РАЗМЕР += 1. Перейти к шагу 2.
6. В данном случае мы имеем первый параметр размера 1 слово и второй параметр размером 2 слова ( ситуация int, long ). Добавляется одно слово выравнивания по смещению РАЗМЕР + 1. В качестве смещения второго параметра взять РАЗМЕР + 3; РАЗМЕР += 3. Перейти к шагу 2.

В результате работы данного алгоритма вычисляется смещение каждого параметра относительно места в стеке, где при вызове функции располагается РС, и общий размер места в стеке занимаемого параметрами.

Например, для функции f из примера 1 параметр a имеет смещение 1, слова заполнения между параметрами a и b имеет смещение 2, параметр b - смещение 4, параметр c - смещение 5. Параметры занимают 5 слов в стеке (адрес растет вверх):

|                         | Смещение |
|-------------------------|----------|
| Текущий указатель стека |          |
| PSW                     |          |
| PC                      | 0        |
| a                       | 1        |
| Неиспользуемое место    | 2        |
| Старшая часть b         | 3        |
| Младшая часть b         | 4        |
| c                       | 5        |

Алгоритм 2. Заполнение стека параметрами при вызове функции:

1. Составляем полный список параметров функции, включая неявно добавленные компилятором
2. Вычисляем размер, занимаемый параметрами функции в стеке, и смещения параметров. Вычисление производится с помощью алгоритма 1.
3. Если размер, занимаемый параметрами в стеке, нечетный, то выравниваем вершину стека на нечетный адрес. Если размер, занимаемый параметрами в стеке, четный, то выравниваем вершину стека на четный адрес.
4. Заносим параметры в стек, добавляя в нужных местах слова заполнения.
5. Производим вызов функции. Шаг 3 гарантирует нам, что вершина стека перед вызовом функции будет иметь четной адрес.

### **3 Возвращаемые функцией значения**

Функция возвращает значение в регистрах. Значения типов - char, short, int, float возвращаются в регистре gr7. Значения типов long, double, long double возвращаются в регистрах gr6, gr7, причем в регистре gr6 находится старшая часть значения, а в регистре gr7 - младшая. Значения указательных типов (преобразуемых к void \*) возвращаются в регистре ar5. Если функция возвращает значение структурного типа, то в регистре ar5 возвращается указатель на структуру. Следует отметить, что возвращаемая структура может располагаться в стеке как ниже, так и выше вершины стека.

#### ***4 Используемые и сохраняемые при вызове функции регистры***

Соглашение о вызове функций регламентирует обращение только с регистрами ar0-ar7, gr0-gr7:

- Значения регистров ar0-ar4, ar6, gr0-gr5 сохраняются.
- Значения регистров ar5 и gr7 не сохраняются.
- Значение регистра gr6 сохраняется в том случае, если он не используется для возврата части длинного значения.
- Регистр ar7 или sp имеет специальное значение указателя на вершину системного стека и после возврата из функции должен иметь прежнее значение.

Действия функций с другими регистрами (не входящими в число ar0-ar7, gr0-gr7) не регламентируются данным соглашением.